PowerShell y logging de seguridad en Windows: visibilidad de la fuerza bruta y del abuso de scripts

Deiner Brenes Soto y Warner Mora Taisigue

Ingeniería del Software, Universidad Técnica Nacional

ISW-612 Administración de Sistemas Operativos de Red

Prof. Marcos Castro

26 de octubre de 2026

I Resumen

En el presente proyecto estudiamos qué tanta visibilidad ofrece los eventos de seguridad de Windows y el registro de PowerShell para detectar ataques de fuerza bruta y abuso de scripts y que cambio ocurre en dicha visibilidad al activar la auditoría. Trabajamos en una máquina virtual (VirtualBox) propia con Windows 11 Enterprise LTSC y PowerShell 5.1. Desarrollamos dos scripts, el primero para consultar, activar y revertir la política de auditoría avanzada y el registro de scripts, mientras que el segundo para recolectar los eventos 4624, 4625, 4688, 4104 y 4740 con `Get-WinEvent` y guardarlos en un archivo JSON. Finalmente, el archivo se analiza en una app web que aplica la regla “5 fallos en 2 minutos”. Simulamos intentos fallidos de inicio de sesión contra una cuenta de prueba en dos configuraciones. Por defecto, cuatro de los siete tipos de visibilidad estudiados estaban desactivados y solo 10 de 16 intentos quedaron registrados como evento 4625, la activar la auditoría se registraron 13 de 13. La regla de la aplicación detectó ambas simulaciones.

II Introducción

Ante un incidente en un equipo con Windows, lo primero que se revisa son los registros de eventos. El problema con esto es que Windows guarda únicamente lo que fue configurado para guardar y muchos de los eventos más útiles vienen desactivados (Kent y Souppaya, 2006; Microsoft, 2021, 2022)

Pregunta de investigación

¿Qué visibilidad dan los eventos de Windows y el logging de PowerShell para detectar fuerza bruta y abuso de scripts?

Hipótesis

Creemos que con Windows por defecto solo vamos a ver los inicios de sesión (4624 y 4625), y que casi nada de lo que pasa con procesos y scripts (4688 y 4104) va a quedar registrado. Al activar la auditoría y el logging de PowerShell deberían aparecer los cuatro eventos, y una regla simple como “5 fallos en 2 minutos” debería detectar una fuerza bruta simulada. Lo esperábamos porque, según Microsoft, parte de estos registros se activa por política, aunque había que confirmarlo en la VM.

III Objetivos

Objetivo general. Medir cuánto cambia la visibilidad de una fuerza bruta simulada entre Windows 11 por defecto y Windows 11 con la auditoría y el registro de PowerShell activados.

Objetivos específicos:

1. Identificar qué eventos registra Windows 11 por defecto y cuáles hay que activar.
2. Crear un script reversible que active la auditoría y otro que recolecte los eventos en JSON.
3. Simular una fuerza bruta en una VM propia y comparar lo registrado en ambas configuraciones.
4. Mostrar los resultados en una aplicación web sencilla, con una regla de detección configurable.

IV Marco teórico

Auditoría avanzada y eventos estudiados

Windows decide qué eventos de seguridad escribe en el registro **Security** según su política de auditoría. Esta política se divide en subcategorías, como *Logon* o *Process Creation*, y

cada una puede registrar aciertos, errores, ambos o nada. La herramienta `auditpol` permite consultarla, cambiarla, respaldarla y restaurarla desde la línea de comandos (Microsoft, s. f.). La

Tabla 1

Eventos de Windows estudiados y su origen

ID	Qué indica	Registro	Subcategoría o política	Fuente
4624	Inicio de sesión exitoso	Security	Logon, aciertos	Microsoft (2026a)
4625	Inicio de sesión fallido	Security	Logon, errores; y Account Lockout, errores	Microsoft (2026a, 2021b)
4740	Cuenta de usuario bloqueada	Security	User Account Management, aciertos	Microsoft (2021a)
4688	Se creó un proceso	Security	Process Creation, aciertos; la línea de comandos requiere otra directiva	Microsoft (2022)
4104	Se ejecutó un bloque de script de PowerShell	PowerShell/Operational	Registro de bloques de script (*Script Block Logging*)	Microsoft (2025)

El 4625 es el centro de este trabajo: **Logon** registra los fallos comunes y **Account Lockout** los intentos contra una cuenta ya bloqueada (Microsoft, 2021b, 2026). Su campo **Statu**s indica el motivo: `0xC000006A` es contraseña incorrecta y `0xC0000234`, cuenta bloqueada. En el 4688, la línea de comandos solo aparece si se activa una directiva adicional y puede incluir contraseñas en texto claro (Microsoft, 2022).

Registro de PowerShell y retención

El registro de bloques de script guarda el contenido de cada bloque que PowerShell procesa como evento 4104, y se activa por directiva de grupo o con el valor `EnableScriptBlockLogging` en el registro de Windows (Microsoft, 2025). Aun sin activarlo, PowerShell registra por su cuenta los bloques cuyo contenido suele aparecer en scripts maliciosos, como un “registro de último recurso” (PowerShell Team, 2015). Como estos registros pueden contener credenciales, Microsoft (2025) recomienda cifrarlos con **Protected**

Event Logging. Además, todo registro tiene un tamaño máximo, cuando se llena, sobrescribe los eventos antiguos y se pierde evidencia (Kent y Souppaya, 2006).

Fuerza bruta y bloqueo de cuentas

MITRE ATT&CK clasifica la fuerza bruta como la técnica T1110 de la táctica de acceso a credenciales, y el adivinar contraseñas como su subtécnica T1110.001 (MITRE, 2026a). En los eventos, este ataque aparece como muchos 4625 contra la misma cuenta o desde el mismo origen en poco tiempo. La defensa habitual es la política de bloqueo, tras varios fallos, Windows bloquea la cuenta por un tiempo. El abuso de PowerShell, por su parte es la técnica T1059.001 (MITRE, 2026b).

V Metodología

Seguimos un diseño que permitió comparar un antes y después, se hizo la misma simulación de fuerza bruta con la configuración por defecto y ya una vez con la auditoría activa.

Tabla 2

Elemento	Valor
Sistema operativo	Windows 11 Enterprise LTSC Evaluation, compilación 26100
PowerShell	Windows PowerShell 5.1.26100.9444
Equipo	W11-WM, máquina virtual propia, sin unirse a dominio
Umbral de bloqueo de cuenta	10 intentos (valor por defecto, observado en ambas pruebas)

Procedimiento

1. Tomamos una instantánea de la VM y guardamos el estado inicial con `-Accion Estado`.
2. Activamos la versión 0.1.0: **Logon, Process Creation**, línea de comandos y registro de scripts, **Account Lockout** quedó por defecto (solo aciertos).
3. **Prueba 1:** ejecutamos `runas /user:prueba_isw612 cmd` 16 veces con una contraseña incorrecta.

4. Notamos que faltaban los intentos posteriores al bloqueo, revertimos todo y corregimos el script.
5. **Prueba 2:** con la versión 0.2.0, que agrega Account Lockout y User Account Management, repetimos la simulación con 13 intentos.
6. Recolectamos la última hora de eventos (ambas pruebas) y cargamos el JSON en la aplicación.
7. Revertimos la configuración, borramos la cuenta y comprobamos que el estado final era igual al inicial.

Las dos pruebas solo difieren en **Account Lockout**, que es justo lo que queríamos comparar. Cada intento quedó registrado como un 4688 de `runas.exe`, y así pudimos contar los intentos reales sin depender del 4625.

Se midieron cuatro cosas, si cada tipo de evento se registra o no en cada configuración, qué parte de los intentos reales quedó como 4625, si la regla “5 fallos en 2 minutos” de la aplicación genera una alerta para cada prueba, y si el estado final del equipo es igual al inicial.

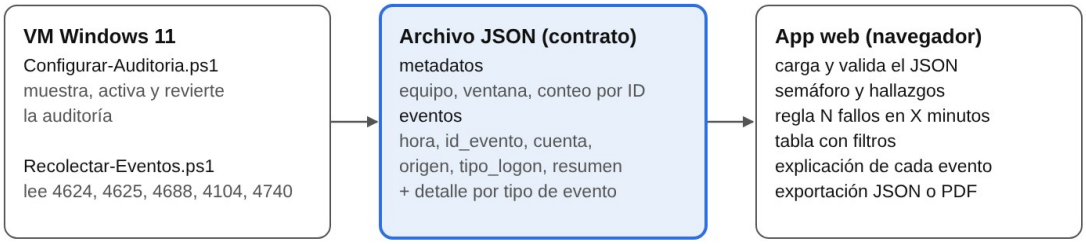
Todo se logró en una máquina virtual propia y aislada, sin malware ni herramientas de ataque, los intentos se escribieron a mano contra una cuenta creada solo para la prueba.

VI Implementación

La solución tiene tres piezas (Figura 1): los scripts producen los datos en la VM, el JSON es el contrato entre las piezas y la aplicación web los presenta en el navegador, sin que nada salga del equipo.

Figura 1

Arquitectura de la solución



Scripts y archivo JSON

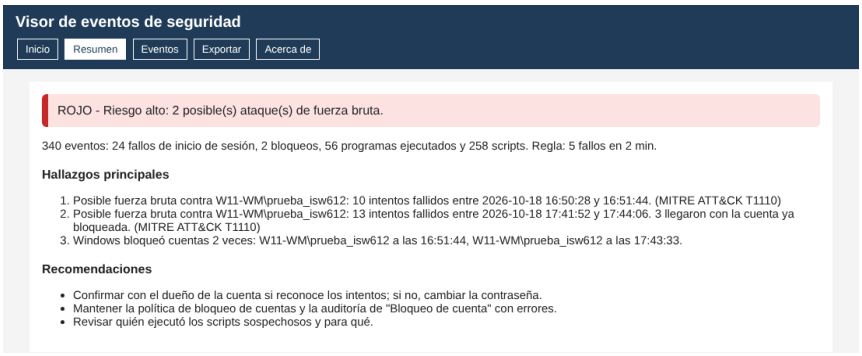
`Configurar-Auditoria.ps1` tiene tres acciones. **Estado** traduce la configuración actual a una lista de qué eventos se pueden ver. **Activar** guarda un respaldo con `auditpol /backup` antes de cambiar nada, y **Revertir** lo restaura con `auditpol /restore` (Microsoft, s. f.). `Recolectar-Eventos.ps1` solo lee: consulta cada evento con `Get-WinEvent -FilterHashtable` (Microsoft, 2026b).

Cada evento se guarda con los seis campos mínimos del enunciado (`hora`, `id_evento`, `cuenta`, `origen`, `tipo_logon` y `resumen`) más un bloque `detalle esto` para lograr que sea fácil de entender por cualquier persona, es decir, cambiamos lenguaje técnico por uno más natural, por ejemplo: `0xc0000234` es “Cuenta bloqueada” y las contraseñas que aparezcan después de `-Password` o `net user` se reemplazan por `[OCULTO]`.

Aplicación web

La aplicación es una página HTML con JavaScript, no necesita de servidor. Tiene cinco pantallas: Inicio el cual carga del JSON o de los datos de ejemplo, Resumen: semáforo, tres hallazgos principales y recomendaciones; Figura 2, Eventos: tabla con filtros por tipo, cuenta y origen, y la explicación de cada evento, Exportar: Permite exportar ya sea JSON o PDF y Acerca de : para la metodología y fuentes.

Figura 2



La regla agrupa los 4625 por cuenta y revisa si hay N fallos dentro de X minutos. Su núcleo es una sola comparación:

```
if (f[i + n - 1].t - f[i].t <= ventana) { /* alerta */ }
```

El semáforo queda en rojo con alguna alerta, en amarillo si solo hay bloqueos o scripts sospechosos y en verde en otro caso. El JSON se muestra con `textContent` y la página no usa internet; con archivos vacíos, dañados o con HTML siempre mostró un mensaje claro sin ejecutar código.

VII Resultados

La recolección final produjo 340 eventos: 258 del 4104, 56 del 4688, 24 del 4625, 2 del 4740 y ninguno del 4624.

Visibilidad por defecto frente a auditoría activa

Tabla 3

Visibilidad de cada tipo de evento antes y después de activar la auditoría

Evento o dato	Qué registra	Por defecto	Con auditoría
4624	Inicio de sesión exitoso	Sí	Sí
4625	Inicio de sesión fallido	Sí	Sí
4625 “cuenta bloqueada”	Intentos contra una cuenta ya bloqueada	No	Sí
4740	Momento del bloqueo	Sí	Sí
4688	Programas ejecutados	No	Sí
4688 con línea de comandos	Parámetros de cada programa	No	Sí
4104	Contenido de los scripts	No (solo sospechosos)	Sí
Tamaño de Security / PowerShell	Espacio antes de sobrescribir	20 / 15 MB	128 / 64 MB

Notat: Datos de `evidencia/estado-antes.json` y `evidencia/estado-despues.json`.

Por defecto, cuatro de los siete tipos de visibilidad estaban desactivados. El script los activó todos y *Revertir* devolvió el equipo a un estado idéntico al inicial.

Simulación de fuerza bruta

Tabla 4

Intentos registrados en cada prueba

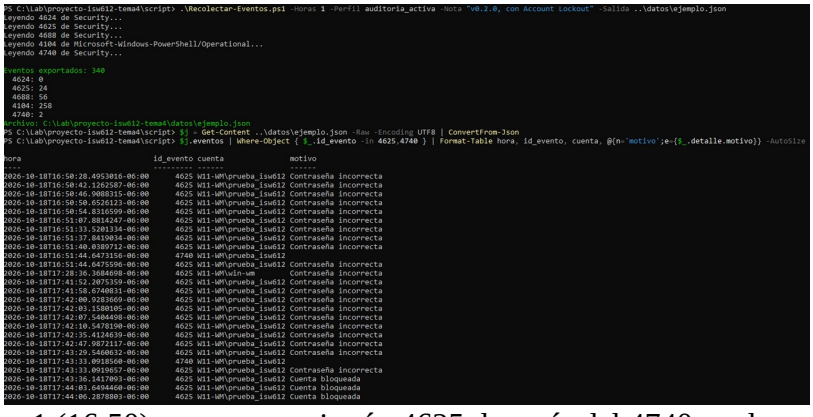
Medida	Prueba 1 (Account Lockout por defecto)	Prueba 2 (Account Lockout con errores)
Intentos reales (4688 de `runas.exe`)	16	13
4625 “Contraseña incorrecta” (0xc000006a)	10	10
Hora del 4740 (cuenta bloqueada)	16:51:44	17:43:33
Intentos después del bloqueo	6	3
4625 “Cuenta bloqueada” (0xc0000234)	0	3
Intentos visibles en 4625	10 de 16	13 de 13

Nota: Datos de `datos/ejemplo.json`.

En ambas pruebas la cuenta se bloqueó en el décimo fallo. Con la configuración por defecto, los 6 intentos posteriores no dejaron ningún 4625: el 37,5 % del ataque fue invisible. Con *Account Lockout* auditando errores, los 3 intentos posteriores sí aparecieron con el motivo “Cuenta bloqueada” (Figura 3).

Figura 3

Secuencia de eventos 4625 y 4740 en ambas pruebas



Nota: En la prueba 1 (16:50) no aparece ningún 4625 después del 4740; en la prueba 2 (17:41) aparecen tres.

Regla de detección

Al cargar el JSON en la aplicación con la regla de 5 fallos en 2 minutos, el semáforo quedó en rojo y aparecieron dos alertas contra la cuenta de prueba: una con 10 intentos entre 16:50:28 y 16:51:44, y otra con 13 intentos entre 17:41:52 y 17:44:06. La segunda avisó, además, que 3 intentos llegaron con la cuenta bloqueada (Figura 2). Con 2 minutos, ambas pruebas se detectan con un umbral de hasta 10 fallos y ninguna desde 12.

VIII Discusión

La hipótesis planteada se cumplió, por defecto se vieron los inicios de sesión, pero no los procesos ni los scripts, ya con la auditoría activa aparecieron los cuatro eventos y la regla detectó ambas simulaciones sin falsas alarmas.

Aunque, lo más importante, no estaba en la hipótesis: un punto ciego en el propio 4625. Los intentos contra una cuenta ya bloqueada los registra la subcategoría Account Lockout (Microsoft, 2021), que por defecto solo audita aciertos aunque solo genera errores. Así, el bloqueo detiene el ataque pero esconde que el atacante siguió intentando.

Limitaciones y mejoras

Usamos una sola VM y solo intentos locales con `runas`, escritos a mano y más lentos que un ataque real. Además, no activamos Protected Event Logging. Como parte de una posible mejora, repetiríamos la prueba desde otra VM para generar inicios de sesión de red y activaríamos ese cifrado.

IX Conclusiones

1. Windows 11 registra por defecto lo suficiente para notar una fuerza bruta (4625 y 4740), pero no para medirla en su totalidad en la prueba se perdió el 37,5 % de los intentos.

2. Por defecto no queda registro de los programas ni de los scripts ejecutados, la evidencia clave del abuso de PowerShell.
3. Con la auditoría activa se vio el 100 % de los intentos, la regla de 5 fallos en 2 minutos detectó ambas simulaciones y el cambio se pudo revertir.

X Referencias

Kent, K. y Souppaya, M. (2006). *Guide to computer security log management* (NIST Special Publication 800-92). National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-92.pdf

Microsoft. (2018, 2 de noviembre). *Account lockout threshold*. Microsoft Learn. https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/security-policy-settings/account-lockout-threshold

Microsoft. (2021a, 7 de septiembre). *4740(S): A user account was locked out*. Microsoft Learn. https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-4740

Microsoft. (2021b, 6 de septiembre). *Audit Account Lockout*. Microsoft Learn. https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/audit-account-lockout

Microsoft. (2022, 24 de enero). *4688(S): A new process has been created*. Microsoft Learn. https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-4688

Microsoft. (2025, 29 de septiembre). *about_Logging* (PowerShell 5.1). Microsoft Learn. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_logging?view=powershell-5.1

Microsoft. (2026a, 29 de marzo). *4625(F): An account failed to log on*. Microsoft Learn. https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-4625

Microsoft. (2026b, 12 de agosto). *Get-WinEvent* (PowerShell 5.1). Microsoft Learn. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.diagnostics/get-winevent?view=powershell-5.1

Microsoft. (s. f.). *auditpol*. Microsoft Learn. Recuperado el 20 de octubre de 2026, de https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/auditpol

MITRE. (2026a, 12 de mayo). *Brute force* (T1110). MITRE ATT&CK. https://attack.mitre.org/techniques/T1110/

MITRE. (2026b, 18 de septiembre). *Command and scripting interpreter: PowerShell* (T1059.001). MITRE ATT&CK. https://attack.mitre.org/techniques/T1059/001/

PowerShell Team. (2015, 9 de junio). *PowerShell ♥ the Blue Team*. PowerShell Team Blog, Microsoft. https://devblogs.microsoft.com/powershell/powershell-the-blue-team/